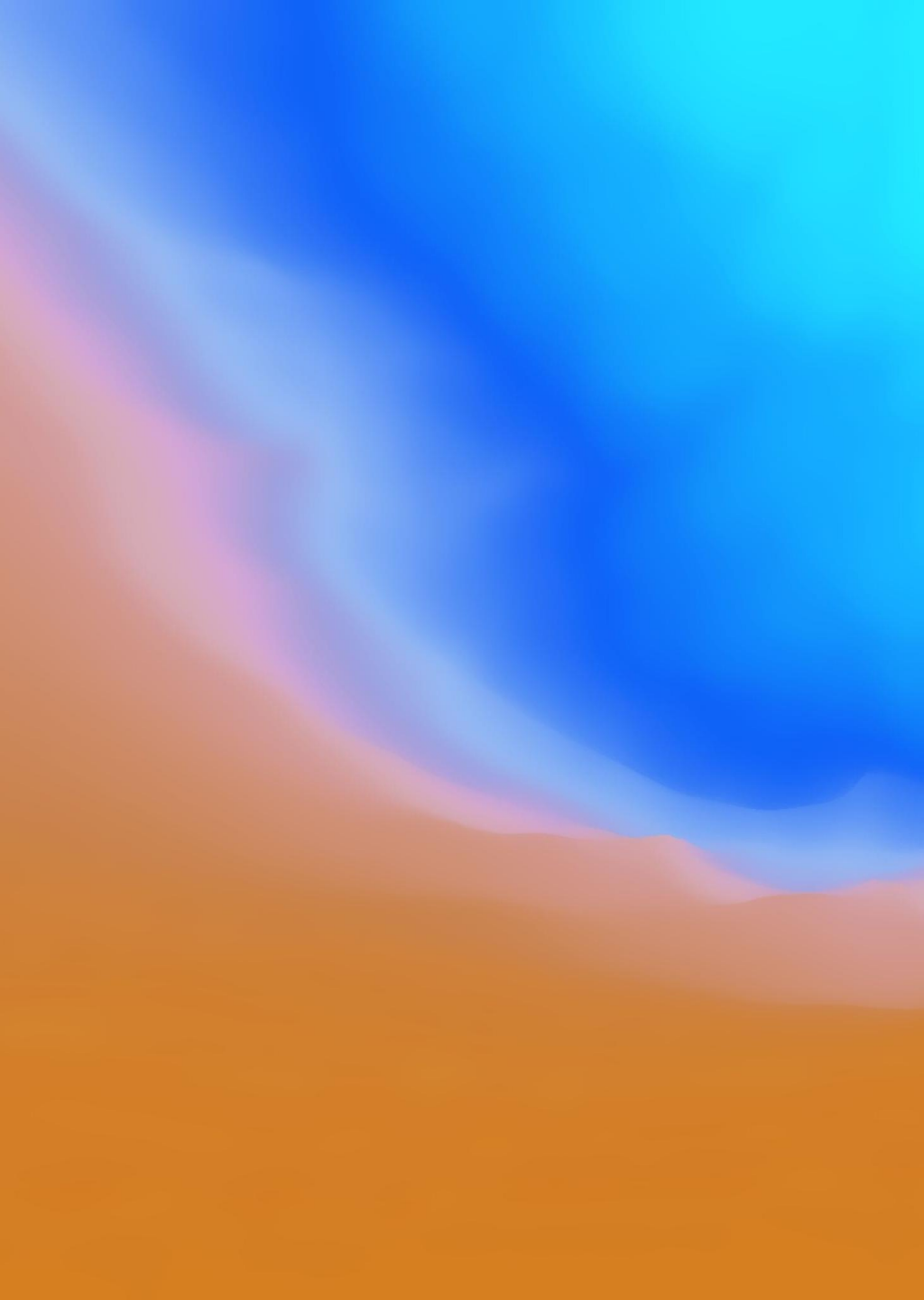




MIDTERM MASTERY

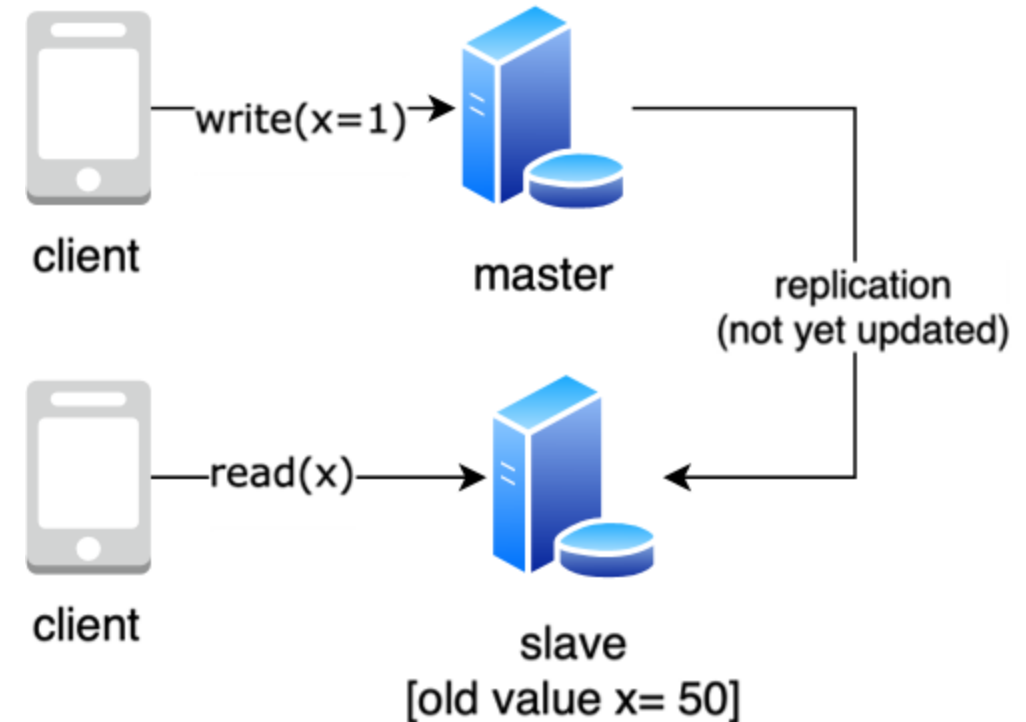
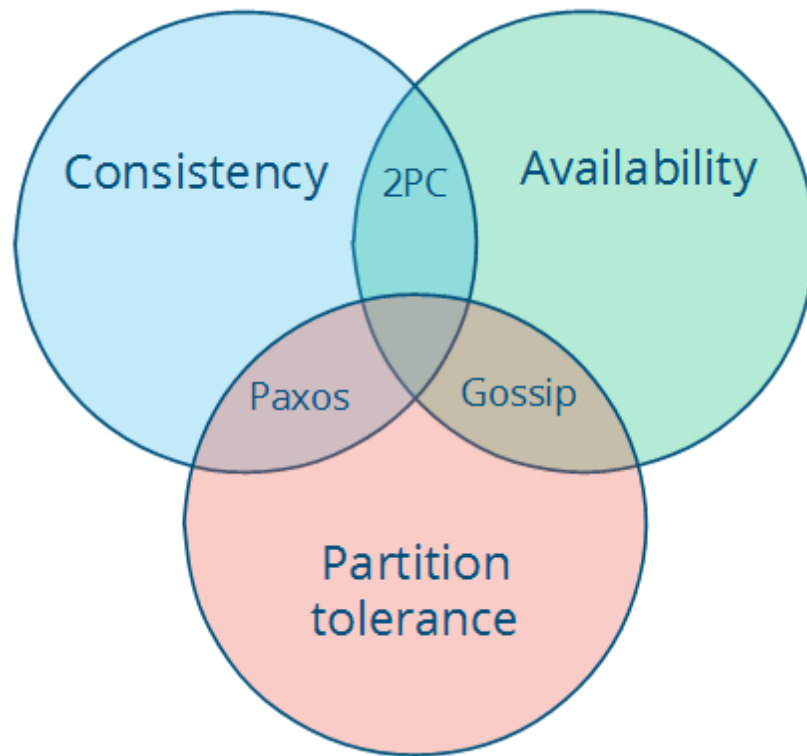
WHAT I HAVE ENJOYED LEARNING SO FAR

CAP Theorem Explains Master-Slave Read Inconsistency

MapReduce: From Paper to Practice

Auto-Scaling in Product-API

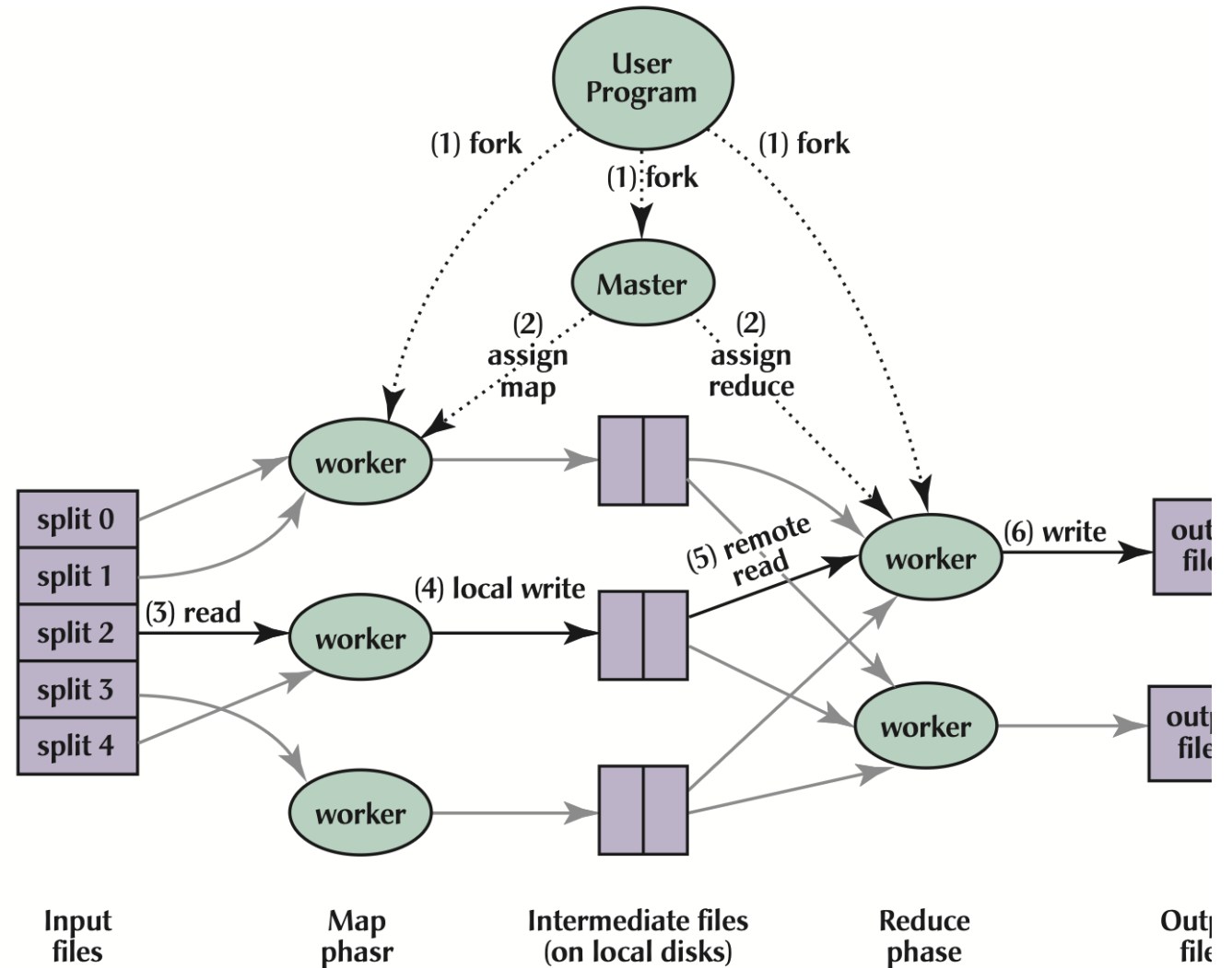
CAP THEOREM EXPLAINS MASTER-SLAVE INCONSISTENCY



MAPREDUCE: FROM PAPER TO PRACTICE

MapReduce = Simple
Abstraction + Powerful
Framework

- Map: Transform data in parallel
- Reduce: Aggregate results
- Framework: Handles failures, scheduling, distribution



MAPREDUCE: FROM PAPER TO PRACTICE

Services	Tasks	Infrastructure	Metrics	Scheduled tasks	Configuration	Tags
Tasks (8)						
Last updated September 28, 2025, 22:38 (UTC-7:00) Manage tags Stop Run new task						
Filter tasks by property or value Filter desired status Running Filter launch type Any launch type < 1 > ⚙						
☐	Task	Last status	Desired st...	Task definition	Health sta...	Created at
☐	 10919549d0af4db1a191...	Running	Running	mapper-task:5	Unknown	7 minutes ago
☐	 12a8ac871f7b4ac896a8d...	Running	Running	mapper-task:5	Unknown	7 minutes ago
☐	 2c22b7746030489eae3a...	Running	Running	mapper-task:5	Unknown	7 minutes ago
☐	 457bddb5f2b249e6832b...	Running	Running	mapper-task:5	Unknown	7 minutes ago
☐	 85e5d35b17e64396bd54...	Running	Running	splitter-task:4	Unknown	7 minutes ago
☐	 a2c3b9f79d0b484cb572f...	Running	Running	reducer-task:1	Unknown	2 minutes ago
☐	 b6f29971e1794233861b...	Running	Running	mapper-task:5	Unknown	7 minutes ago
☐	 f7c5f3a9928b4b0b86d54...	Running	Running	mapper-task:5	Unknown	7 minutes ago

```
### Perform Map Reduce Steps ###
# 1. Split the input file
split_result = await self.http_get_json(
    f"http://{splitter_ip}:8080/split",
    params={"url": input_url, "chunks": str(chunks)}
)

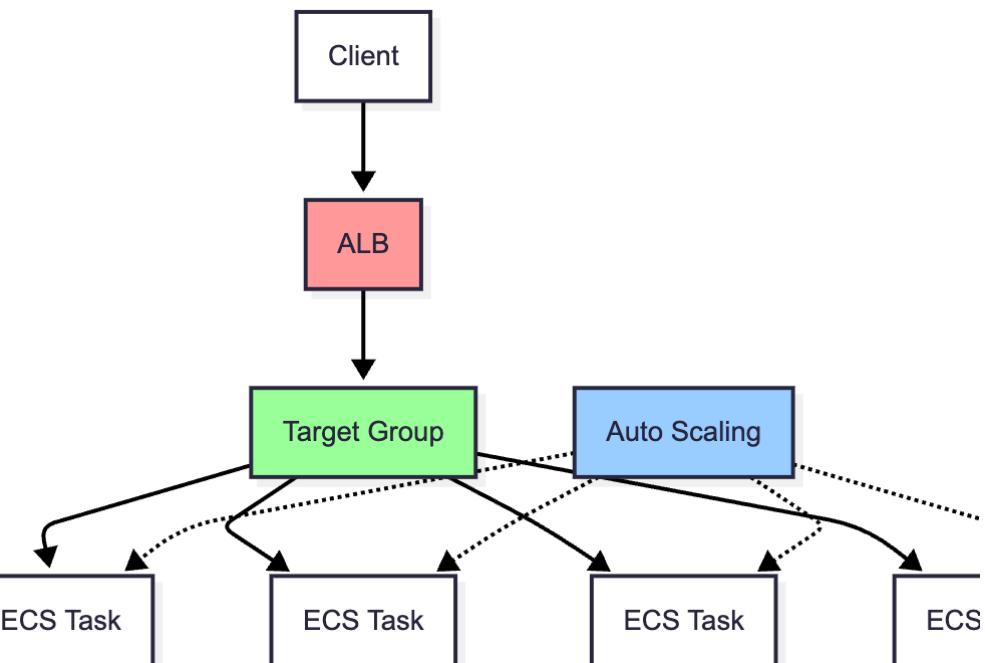
chunk_urls = split_result.get("chunk_urls", [])

# 2. Map each chunk
map_tasks = []
for i, chunk_url in enumerate(chunk_urls):
    mapper_ip = mapper_ips[i % len(mapper_ips)]
    map_tasks.append(self.http_get_json(
        f"http://{mapper_ip}:8080/map",
        params={"url": chunk_url}
    ))

map_results = await asyncio.gather(*map_tasks)
print(f"Completed {len(map_results)} map operations")

# 3. Reduce the mapped results
reduce_result = await self.http_get_json(
    f"http://{reducer_ip}:8080/reduce"
)
```

AUTO-SCALING IMPLEMENTATION

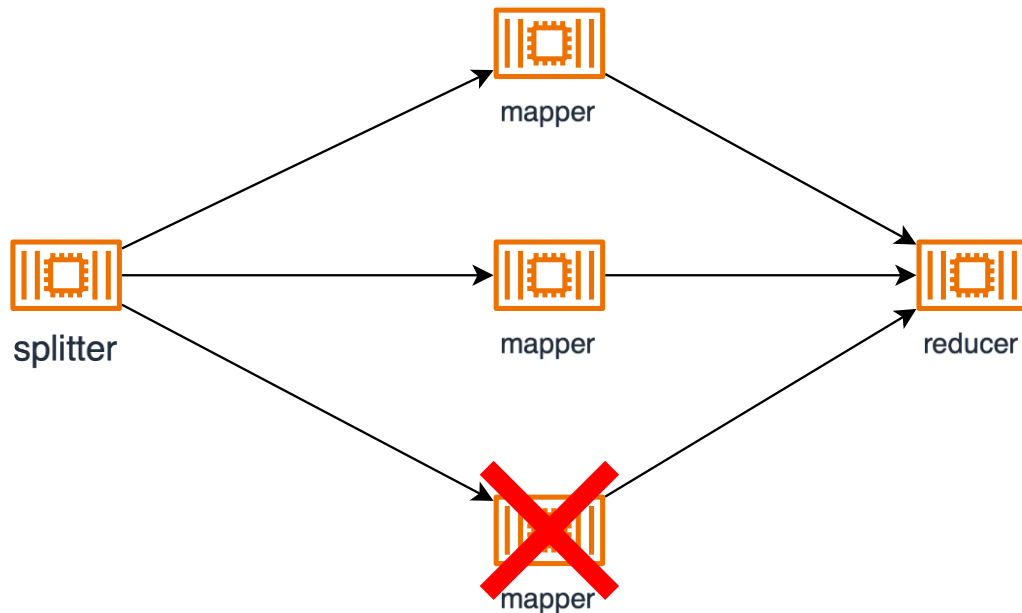


Users	Task Count	CPU Usage (%)	Memory Usage (%)	RPS	50% Response Time (ms)	95% Response Time (ms)
0	2	3	6	N/A	N/A	N/A
20	2	40	10	9	40	200
40	2	70	14	17	70	440
60	3	80	15	25	55	430
80	4	75	18	34	47	340

CRASHING & RECOVERING

MapReduce Fault Tolerance

PARTIAL MAPPER FAILED



```
Downloading text from: s3://mapreduce-experiment-975050147762/50MB-TXT-FILE.txt
Downloaded 52433149 characters
Word counting completed in 1.84 seconds
```

```
Word Count Results:
Total words: 6883544
Unique words: 311
```

```
%2Fchunk_6.txt
Completed 6 map operations
HTTP GET: http://54.213.113.197:8080/reduce
MapReduce completed in 6.73 seconds
MapReduce Result: Successfully reduced all mapper results. Total: 5735753 words, Unique: 311 words
Total Words: 5735753
Unique Words: 311
(venv) wenshuangzhou@wenshuangs-MacBook-Pro CS6650 %
```

FAULT REASON

```
# 2. Map each chunk
map_tasks = []
for i, chunk_url in enumerate(chunk_urls):
    mapper_ip = mapper_ips[i % len(mapper_ips)]
    map_tasks.append(self.http_get_json(
        f"http://{mapper_ip}:8080/map",
        params={"url": chunk_url}
    ))

    You, 3 weeks ago • add experiments

map_results = await asyncio.gather(*map_tasks)
print(f"Completed {len(map_results)} map operations")

# 3. Reduce the mapped results
reduce_result = await self.http_get_json(
    f"http://{reducer_ip}:8080/reduce"
)
```

A QUICK FIX

Completed 0 map operations

HTTP GET: http://54.213.113.197:8080/reduce

MapReduce completed in 9.21 seconds

MapReduce Result: Successfully reduced all mapper results. Total: 6883544 words, Unique: 311 words

Total Words: 6883544

Unique Words: 311

2. Map each chunk

```
map_tasks = []
for i, chunk_url in enumerate(chunk_urls):
    mapper_ip = mapper_ips[i % len(mapper_ips)]
    map_tasks.append(self.http_get_json(
        f"http://{mapper_ip}:8080/map",
        params={"url": chunk_url}
    ))
```

You, 9 hours ago • add midterm part2

```
map_results = await asyncio.gather(*map_tasks)
```

```
for i, result in enumerate(map_results):
    if isinstance(result, dict) and "error" in result:
        alternate_mapper_ip = self._get_alterate_mapper(i)
        if alternate_mapper_ip:
            chunk_url = chunk_urls[i]
            alternative_result = await self.http_get_json(
                f"http://{alternate_mapper_ip}:8080/map",
                params={"url": chunk_url}
            )
```

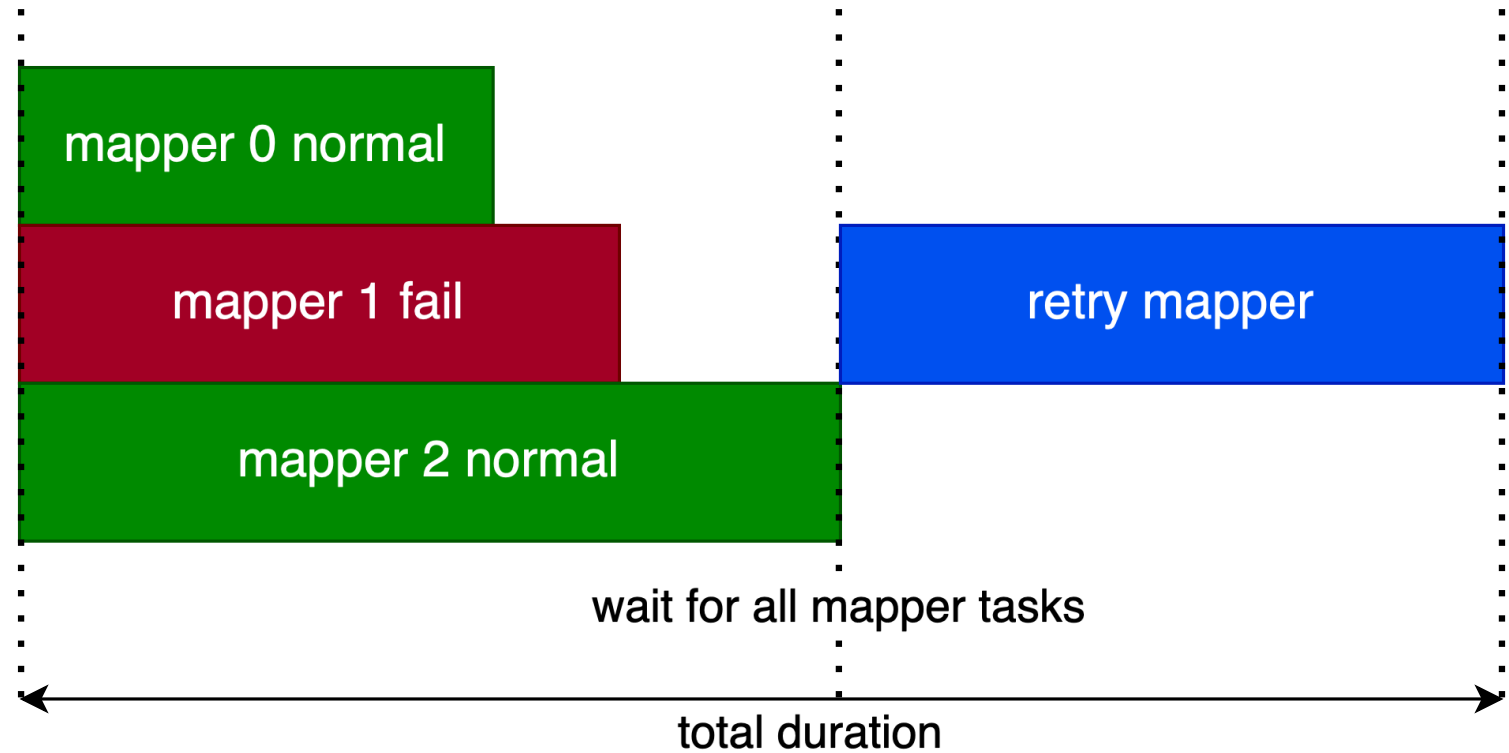
```
print(f"Completed {len(map_results)} map operations")
```

3. Reduce the mapped results

```
reduce_result = await self.http_get_json(
    f"http://{reducer_ip}:8080/reduce"
)
```

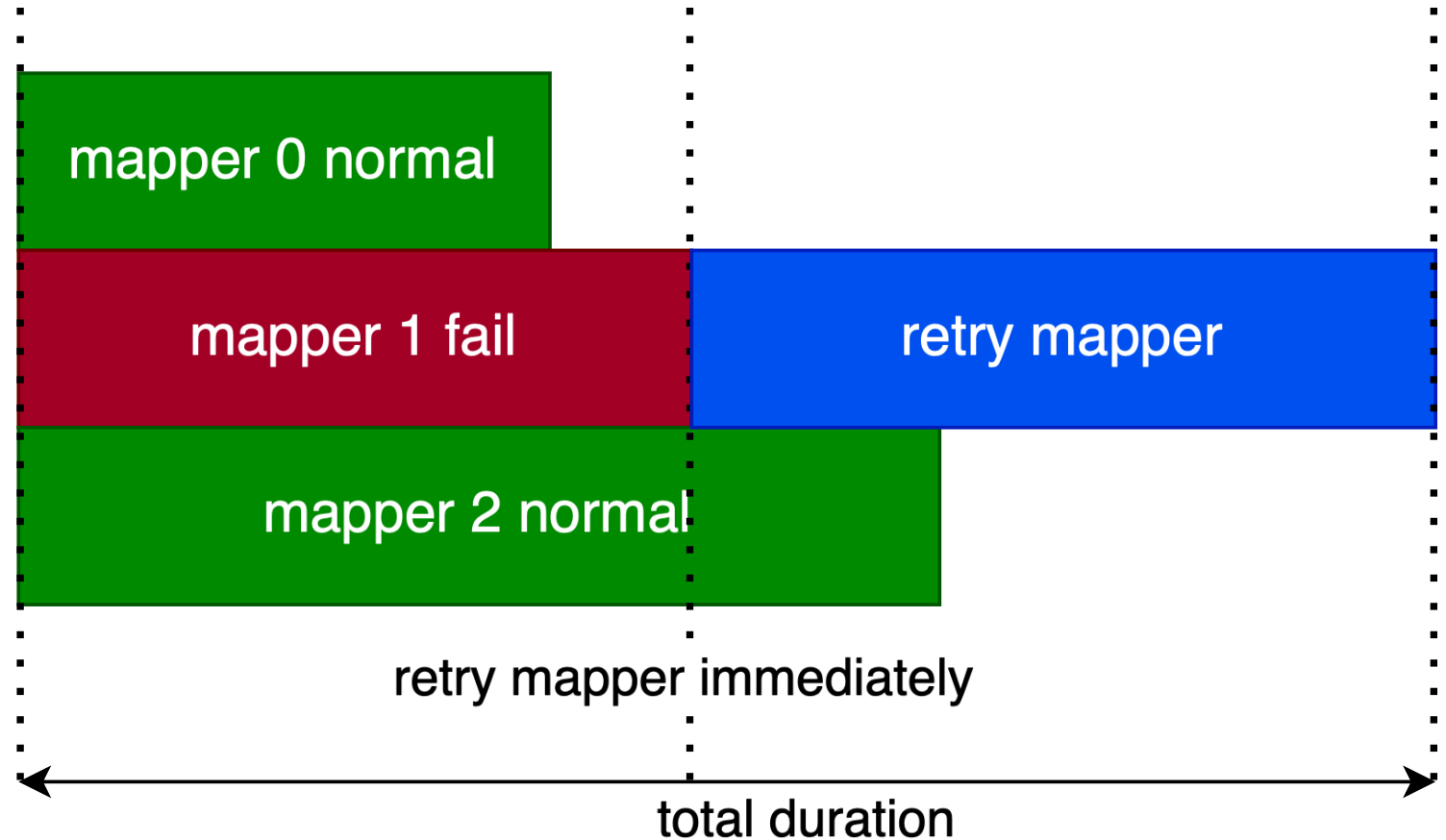
mapper available

ANOTHER PROBLEM



```
Completed 8 map operations
HTTP GET: http://34.220.240.94:8080/reduce
MapReduce completed in 18.78 seconds
MapReduce Result: Successfully reduced all mapper results. Total: 6883544 words, Unique: 311 words
Total Words: 6883544
Unique Words: 311
```

QUICK RESPONSE TO FAILURE HANDLING



```

while len(completed_chunks) < len(chunk_urls):
    if not active_tasks:
        break # No more active tasks to process
    done_tasks, _ = await asyncio.wait(active_tasks.values(), return_when=asyncio.FIRST_COMPLETED)

    for completed_task in done_tasks:
        # Find which chunk this task corresponds to
        chunk_idx = next((idx for idx, task in active_tasks.items() if task == completed_task), None)
        if chunk_idx is None:
            continue

        try:
            result = await completed_task
            if isinstance(result, dict) and "error" in result:
                alternate_mapper_ip = self._get_alternate_mapper(chunk_idx)
                if alternate_mapper_ip:
                    chunk_url = chunk_urls[chunk_idx]
                    retry_task = asyncio.create_task(
                        self.http_get_json(
                            f"http://{alternate_mapper_ip}:8080/map",
                            params={"url": chunk_url}
                        )
                    )
                    active_tasks[chunk_idx] = retry_task
                    continue
            else:
                raise Exception(f"Map operation failed and no alternate mapper available")
        except:
            # Completed 6 map operations
            # HTTP GET: http://34.220.240.94:8080/reduce
            # MapReduce completed in 9.67 seconds
            # MapReduce Result: Successfully reduced all mapper results. Total: 6883544 words, Unique: 311 words
            # Total Words: 6883544
            # Unique Words: 311
            # (venv) yenchuanzhu@yenchuanzhu-MacBook-Pro:CS6650 %
            completed_chunks.add(chunk_idx, result)
            del active_tasks[chunk_idx]

```

QUICK RESPONSE TO FAILURE HANDLING
